HBTXR: An Algorithm–Hardware Co-Designed Hybrid Eye Tracking System for On-Device Extended Reality

Anonymous Author(s)

***Abstract*—Real-time eye tracking for extended reality (XR) must simultaneously satisfy semantic robustness, low latency, and deployment efficiency. Existing frame-based methods provide stable visual cues but incur dense sensing and high memory traffic, whereas event-based methods offer high temporal resolution yet remain weaker in relocalization and long-horizon stability. Prior hybrid approaches partially bridge this gap, but often treat model fusion, runtime switching, and deployment mapping as loosely coupled steps. This paper presents HBTXR, an algorithm–hardware co-designed hybrid eye-tracking system that explicitly unifies frame-guided Search and event-guided Track for on-device XR. HBTXR integrates four tightly coupled components: 1) a geometry-stable hybrid supervision framework that canonicalizes frame, event, mask, and ellipse targets under a common training contract; 2) a hybrid Search–Track transformer with modality-specific embedding paths, a shared backbone, and decoupled Search, Event, Track, and Mask heads; 3) a runtime-aware Search/Track scheduler that switches between refresh-oriented and state-persistent execution according to confidence, similarity, event density, and eye-state signals; and 4) a mode-aware FPGA deployment architecture with a shared transformer engine, reuse-oriented dataflow, and FSM-based runtime control. Experimental results show that HBTXR achieves a pupil-center error of 0.42 pixels, a gaze error of 0.68°, and an end-to-end latency of 0.57 ms in Track mode, while maintaining a Search success rate of 97.8%. These results indicate that HBTXR provides a practical operating point for robust, low-latency, and deployment-oriented eye tracking in next-generation XR systems.**

***Index Terms*—Eye tracking, extended reality, event camera, hybrid sensing, transformer accelerator, FPGA, on-device inference.**

I. Introduction

Eye tracking is becoming a foundational capability in extended reality (XR), enabling gaze-based interaction, foveated rendering, attention-aware interfaces, and privacy-preserving on-device perception [1], [2]. In practical head-mounted XR systems, however, eye tracking is constrained by three requirements that must be met simultaneously: it must remain semantically robust under appearance variation, temporally responsive under fast eye motion, and efficient enough to operate under tight power, memory, and form-factor budgets. These requirements make on-device XR eye tracking a distinctly system-level problem rather than a stand-alone vision task.

Prior work has mainly advanced eye tracking along three directions: frame-based estimation, event-based estimation, and hybrid event–frame processing. Frame-based methods provide semantically rich observations and strong global recovery, but repeated dense-image processing introduces substantial bandwidth, compute, and latency overhead [3], [4], [8]. Event-based methods exploit asynchronous brightness changes to realize sparse and low-latency update [5]–[7], [9]–[15], yet they are more vulnerable to drift, weak-event regimes, and loss of a global anchor. Hybrid methods combine both modalities and improve the balance between semantic stability and temporal responsiveness [16], but many existing designs still optimize fusion at the model level while leaving supervision, runtime mode switching, and deployment mapping only weakly coupled.

This gap is increasingly problematic for XR deployment. In real use, relocalization and local update do not play the same role. Relocalization requires broad semantic context and must tolerate degraded states such as drift, occlusion, low event density, or eye closure. In contrast, steady-state tracking should be lightweight, state-persistent, and optimized for short-horizon residual motion. Treating these two operating regimes as one undifferentiated multimodal regression task leaves both algorithmic efficiency and hardware efficiency on the table.

To address these issues, we propose **HBTXR**, an algorithm–hardware co-designed hybrid eye-tracking system for on-device XR. As illustrated in Fig. 1, HBTXR establishes an integrated pipeline from geometry-stable supervision and hybrid model design to runtime scheduling and accelerator mapping. Rather than combining frame and event inputs naively, HBTXR explicitly separates *Frame Search* and *Event Track*. Search performs robust relocalization and anchor refresh from semantically stable frame cues. Track performs event-guided residual update conditioned on the previous state. This algorithmic decomposition is then reflected in the runtime scheduler and the hardware execution path, enabling mode-aware reuse and low-overhead control.

At the algorithm level, HBTXR introduces a canonical supervision contract that aligns frame, event, mask, and ellipse targets under one geometry. At the runtime level, it exposes explicit Search and Track states that allow the system to refresh only when necessary. At the hardware level, it maps these two modes onto a shared accelerator substrate with different buffering and execution granularities. In this way, HBTXR internalizes deployment constraints during model design and, conversely, shapes the hardware around the actual operating semantics of the hybrid tracker.

The key contributions of this work are summarized below.

- *Geometry-stable hybrid supervision:* We propose a canonical supervision framework that aligns frame-based relo-

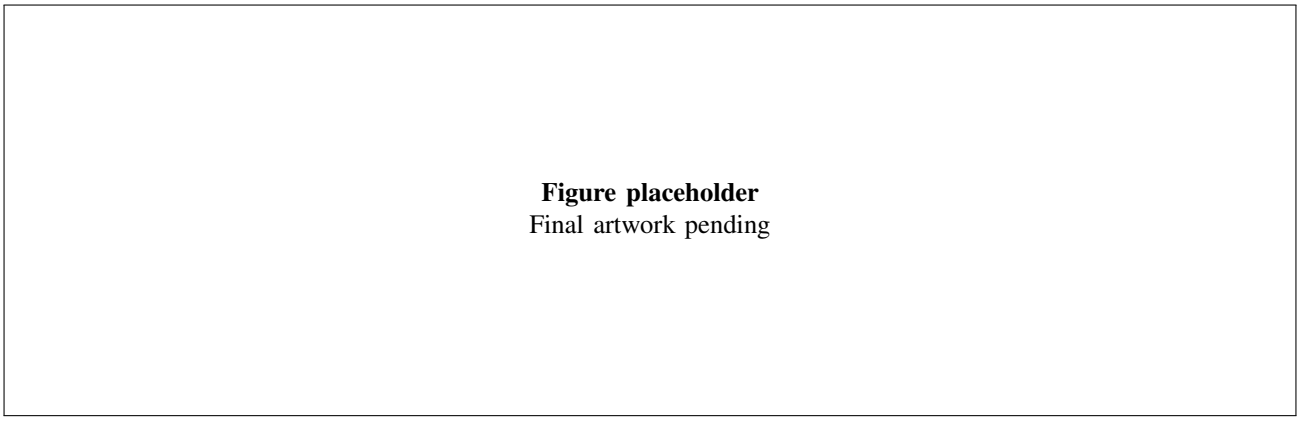

Fig. 1. Overview of HBTXR. The proposed framework unifies geometry-stable supervision, a hybrid Search–Track transformer, a runtime-aware scheduler, and a mode-aware accelerator architecture for on-device XR eye tracking.

calization and event-based residual update under a shared ellipse-centric target representation.

- *Hybrid Search–Track transformer:* We design a shared-backbone transformer with decoupled Search, Event, Track, and Mask heads, enabling robust relocalization and low-latency update within one hybrid model.
- *Runtime-aware Search/Track scheduling:* We develop an explicit scheduler that switches between refresh-oriented and state-persistent modes using confidence, similarity, event density, and eye-state cues.
- *Mode-aware deployment architecture:* We present a mixed-granularity accelerator mapping that exploits feature reuse, shared backbone execution, and lightweight control for on-device XR operation.

The rest of this paper is organized as follows. Section II reviews related work and motivates HBTXR. Section III presents the proposed hybrid eye-tracking algorithm. Section IV describes the accelerator architecture. Section V reports experimental results, comparisons, and ablations. Section VI concludes the paper.

II. Related Work and Motivation

A. Frame-Based Eye Tracking

Frame-based eye tracking has long been the dominant approach in head-mounted and near-eye systems [3], [4], [8]. These methods infer pupil location, iris geometry, or gaze direction from semantically rich RGB or infrared observations, which makes relocalization and calibration-aware regression relatively straightforward. In practice, this often translates to strong global robustness under moderate pose and appearance variation.

The main limitation of frame-based pipelines is their dense processing cost. Repeated acquisition and full-frame inference incur substantial bandwidth and memory traffic, and their effective temporal responsiveness is bounded when rapid eye motion must be tracked continuously. As a result, frame-only methods remain strong for semantic recovery but weak as a stand-alone solution for low-latency XR deployment.

B. Event-Based Eye Tracking

Event-based eye tracking exploits the asynchronous sensing principle of event cameras to realize sparse, high-temporal-resolution, and low-latency processing [5]–[7], [9]–[15], [17], [18]. Representative methods such as E-Track estimate pupil states from event representations [6], while FACET demonstrates that geometry-aware event modeling can achieve both accuracy and speed [7]. Other works improve performance with anti-blink mechanisms, sparse computation, adaptive slicing, or spiking inference [11], [17], [19].

Despite these advantages, event-only pipelines remain weaker in semantic stabilization and re-initialization. When event density drops or drift accumulates, the tracker may lose the global anchor needed for reliable recovery. This makes event-based methods attractive for short-horizon updates but less sufficient as a complete stand-alone XR solution.

C. Hybrid Event–Frame Eye Tracking

Hybrid eye tracking combines the semantic stability of frames and the temporal responsiveness of events [16], [20]. EX-Gaze is a representative example that couples frame-based relocalization with event-guided update for high-frequency on-device tracking [16]. EV-Eye further highlights the value of combining semantic frame cues with event-driven motion signals in near-eye settings [20].

However, most existing hybrid approaches still treat multi-modal fusion primarily as a modeling problem. Supervision design, runtime state switching, and deployment architecture are often treated as downstream implementation details. For XR eye tracking, this is a missed opportunity because Search and Track differ not only in modality emphasis, but also in their control flow, state dependency, latency profile, and memory behavior.

D. Deployment-Oriented and Hardware-Aware Eye Tracking

An increasingly important direction focuses on deployment-aware eye tracking and specialized acceleration [21]–[24]. EyeCoD demonstrates optics–algorithm co-design for compact eye-tracking systems [21]. SEE shows that event-based eye tracking can be co-designed with sparse CNNs and FPGA

execution [23]. JaneEye demonstrates low-latency, energy-efficient event-based tracking in ASIC form [22], while Retina explores spiking-hardware deployment [24].

These studies confirm that practical eye tracking must be assessed beyond model accuracy. Memory movement, synchronization overhead, dataflow structure, and mode-specific workload behavior all directly shape end-to-end latency and energy. Yet most deployment-oriented systems target a single sensing path or a fixed inference schedule, leaving limited support for a hybrid Search/Track workload with explicit runtime adaptivity.

E. Motivations for HBTXR

The above limitations motivate HBTXR as a unified hybrid eye-tracking system in which supervision, model structure, runtime scheduling, and deployment mapping are co-designed rather than optimized independently. At the algorithm level, HBTXR formulates eye tracking as Search and Track rather than one monolithic regressor. At the runtime level, it exposes explicit switching signals that can guide refresh and reuse. At the hardware level, it avoids duplicating complete frame and event pipelines by reusing a shared backbone and attaching mode-specific heads. This holistic viewpoint is the core stylistic and methodological shift that differentiates HBTXR from prior isolated solutions.

III. The Proposed HBTXR Algorithm

A. Problem Formulation

As shown in Fig. 2, HBTXR operates under a hybrid sensing setup composed of a frame stream, an event stream, and a recurrent tracking state. Let $I_t \in \mathbb{R}^{H\times W}$ denote the frame acquired at time t, and let

$$E_t = \{e_k = (x_k, y_k, p_k, \tau_k)\}_{k=1}^{N_t} \tag{1}$$

denote the set of events collected over the current temporal window, where (x_k, y_k) is the event location, $p_k \in \{-1, +1\}$ is the polarity, and τ_k is the timestamp. The event stream is converted into a polarity-split event tensor $V_t \in \mathbb{R}^{2\times H\times W}$ by

$$V_t = \Phi(E_t; \omega_t), \tag{2}$$

where ω_t denotes the event-window construction policy, such as fixed-count or time-bin slicing.

The objective is to estimate the current pupil state

$$s_t = [x_t, y_t, a_t, b_t, u_t, v_t] \in \mathbb{R}^6, \tag{3}$$

where (x_t, y_t) is the pupil center, (a_t, b_t) are the ellipse axes, and (u_t, v_t) is a trigonometric orientation code. We adopt

$$[u_t, v_t] = [\sin(2\theta_t), \cos(2\theta_t)], \tag{4}$$

which avoids the wrap-around discontinuity of direct angle regression and is more stable for ellipse modeling.

A central design choice of HBTXR is to formulate hybrid eye tracking as a Search-and-Track problem rather than a single multimodal prediction task. Search is responsible for robust relocalization and anchor refresh using frame-guided semantic cues. Track performs low-latency residual update using event evidence and the previous state. We further include an event-conditioned auxiliary branch to support cross-branch consistency and runtime confidence estimation.

B. Geometry-Stable Hybrid Supervision

A major challenge in hybrid frame–event tracking is that frame cues, event cues, mask cues, and recurrent updates must all be trained against a common geometric target. To address this, HBTXR adopts a geometry-stable supervision contract. Let $\mathcal{T}(\cdot)$ denote a canonicalization operator. Each training sample is transformed as

$$(\bar{I}_t, \bar{V}_t, \bar{s}_{t-1}, \bar{s}_t, \bar{M}_t, \bar{B}_t, \bar{\xi}_t) = \mathcal{T}(I_t, E_t, s_{t-1}, s_t, M_t, B_t, \xi_t), \tag{5}$$

where $\bar{I}_t$ and $\bar{V}_t$ are the canonical frame and event inputs, $\bar{M}_t$ is the pupil mask, $\bar{B}_t$ is the eye-region target, and $\bar{\xi}_t$ gathers annotation confidence, mask validity, track validity, and eye-state metadata.

This contract provides three key benefits. First, Search and Track are optimized against the same target geometry, preventing incompatible drift between branches. Second, mask, ellipse, and state targets remain spatially aligned across modalities. Third, the quality metadata is propagated into both the loss design and the runtime scheduler, enabling reliability-aware optimization rather than uniform treatment of all samples.

C. Hybrid Search–Track Transformer

The proposed HBTXR network is shown in Fig. 2. The model uses modality-specific embedding paths followed by a shared transformer backbone. Instead of fully fusing frame and event tensors at the input level, HBTXR preserves modality-specific front-ends and reuses a common token-processing trunk, allowing Search-oriented frame inference and Track-oriented event inference to coexist efficiently.

For the frame branch,

$$Z_t^f = \mathcal{B}\big(\mathcal{A}_f(\mathcal{P}_f(\bar{I}_t))\big), \tag{6}$$

and for the event branch,

$$Z_t^e = \mathcal{B}\big(\mathcal{A}_e(\mathcal{P}_e(\bar{V}_t))\big), \tag{7}$$

where $\mathcal{P}_f(\cdot)$ and $\mathcal{P}_e(\cdot)$ denote patch embedding, $\mathcal{A}_f(\cdot)$ and $\mathcal{A}_e(\cdot)$ are modality adapters, and $\mathcal{B}(\cdot)$ is a shared Partial DeiT-Tiny backbone.

On top of the shared backbone, HBTXR employs decoupled prediction heads: Eye Region, Pupil Search, Event Search, Pupil Track, Search Mask, and Auxiliary State heads. Let p_t^f and p_t^e denote the pooled frame and event features. The frame search branch predicts

$$\hat{y}_t^s = h_s(p_t^f), \qquad \hat{s}_t^s = \hat{y}_t^s[1{:}6], \tag{8}$$

while the event branch predicts

$$\hat{y}_t^e = h_e(p_t^e), \qquad \hat{s}_t^e = \hat{y}_t^e[1{:}6]. \tag{9}$$

The dense mask branch predicts

$$\hat{M}_t = h_m(Z_t^f), \tag{10}$$

which provides geometric regularization and an auxiliary signal for Search confidence.

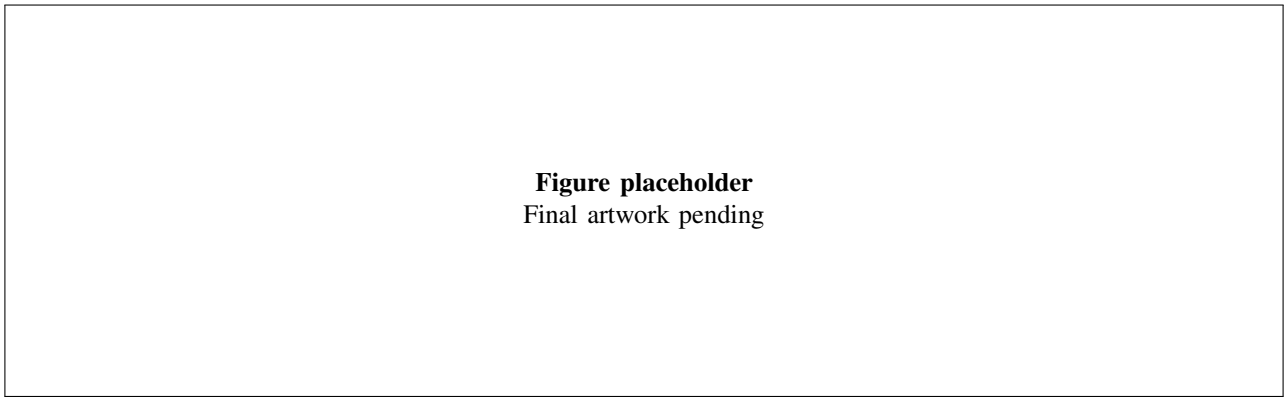

Fig. 2. Architecture of the proposed HBTXR algorithm. Frame and event inputs are processed by modality-specific embedding paths and a shared transformer backbone, followed by decoupled Search, Event, Track, and Mask heads.

D. Search Branch, Track Branch, and State Update

Search is designed for global relocalization and is therefore guided by frame observations. Track is designed for low-latency residual update. It combines the event feature p_t^e with an encoded previous state $\psi(\bar{s}_{t-1})$ and predicts

$$\Delta\hat{r}_t = h_t([p_t^e; \psi(\bar{s}_{t-1})]), \tag{11}$$

where

$$\Delta\hat{r}_t = [\Delta\hat{x}_t, \Delta\hat{y}_t, \Delta\log\hat{a}_t, \Delta\log\hat{b}_t, \Delta\hat{u}_t, \Delta\hat{v}_t, \hat{c}_t^{trk}, \hat{q}_t^{trk}]. \tag{12}$$

The updated tracking state is decoded as

$$\hat{x}_t = x_{t-1} + \Delta\hat{x}_t, \qquad \hat{y}_t = y_{t-1} + \Delta\hat{y}_t, \tag{13}$$

$$\hat{a}_t = a_{t-1}\exp(\Delta\log\hat{a}_t), \qquad \hat{b}_t = b_{t-1}\exp(\Delta\log\hat{b}_t), \tag{14}$$

$$[\hat{u}_t, \hat{v}_t] = \frac{[u_{t-1} + \Delta\hat{u}_t,\ v_{t-1} + \Delta\hat{v}_t]}{\|[u_{t-1} + \Delta\hat{u}_t,\ v_{t-1} + \Delta\hat{v}_t]\|_2 + \epsilon}. \tag{15}$$

This residual parameterization is especially suitable for short-horizon event-driven update because it constrains prediction to a local correction around the current anchor.

E. Runtime-Aware Search/Track Scheduler

A key component of HBTXR is the explicit Search/Track scheduler. Rather than executing all branches uniformly, the scheduler selects either low-latency Track or refresh-oriented Search according to runtime reliability. Let c_t^s be Search confidence, c_t^{trk} tracking confidence, q_t^{trk} tracking quality, ρ_t cross-branch similarity, d_t event density, and o_t a binary eye-state indicator. The scheduler transitions from Search to Track when

$$\text{Search} \rightarrow \text{Track} \quad \text{if } c_t^s > \tau_s. \tag{16}$$

It falls back to Search when the Track path becomes unreliable:

$$\begin{aligned}\text{Track} \rightarrow \text{Search} \quad \text{if } & (c_t^{trk} \le \tau_t) \vee (q_t^{trk} \le \tau_q) \\ & \vee (\rho_t \le \tau_\rho) \vee (d_t \le \tau_d) \vee (o_t = 1).\end{aligned} \tag{17}$$

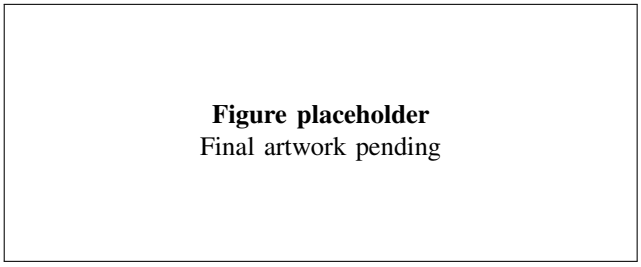

Fig. 3. Runtime-aware Search/Track scheduler. The scheduler switches between refresh-oriented Search and state-persistent Track using confidence, similarity, event density, and eye-state signals.

This scheduler is important not only as a tracking policy but also as an algorithm–hardware contract. It exposes an operating semantics that can be mapped directly to differentiated execution paths in the accelerator.

F. Training Strategy and Loss Design

HBTXR is trained in two stages. The first stage emphasizes Search-oriented relocalization so that the frame branch learns stable global geometry. The second stage jointly fine-tunes Search, Event, and Track supervision with cross-branch consistency. This stage-wise optimization is important because relocalization and residual tracking exhibit different failure modes and optimization dynamics.

The total training objective is

$$\begin{aligned}\mathcal{L} = {} & \lambda_{eye}\mathcal{L}_{eye} + \lambda_{mask}\mathcal{L}_{mask} + \lambda_{search}\mathcal{L}_{search} + \lambda_{event}\mathcal{L}_{event} \\ & + \lambda_{track}\mathcal{L}_{track} + \lambda_{cons}\mathcal{L}_{cons} + \lambda_{ctr}\mathcal{L}_{ctr} + \lambda_{aux}\mathcal{L}_{aux}.\end{aligned} \tag{18}$$

For both Search and Track, we use a geometry-aware term in addition to standard center and axis regression. Let $\mu(s)$ and $\Sigma(s)$ denote the ellipse center and covariance implied by state s. We define

$$\mathcal{L}_{geo}(\hat{s}, s) = \|\mu(\hat{s}) - \mu(s)\|_2^2 + \left\|\Sigma(\hat{s})^{1/2} - \Sigma(s)^{1/2}\right\|_F^2, \tag{19}$$

which stabilizes the full ellipse geometry rather than only supervising the center. Quality-aware gating is further applied based on annotation confidence, mask validity, tracking validity, and eye state.

IV. The Proposed HBTXR Accelerator

A. Design Challenges and Co-Design Principles

The deployment objective of HBTXR is not merely to accelerate a generic transformer. Instead, the accelerator must support a hybrid eye-tracking workload whose operating modes have fundamentally different behaviors. Search performs robust relocalization and anchor refresh using frame-guided features, whereas Track performs event-driven residual update around the previous state. This asymmetry introduces three hardware challenges.

First, Search and Track demand different dataflows. Search is refresh-oriented and requires broader feature regeneration. Track is reuse-oriented and benefits from persistent state and incremental update. A uniform inference path therefore either wastes computation during Track or provides insufficient context during Search. Second, the shared transformer backbone contains synchronization-heavy stages, especially during token interaction and branch coordination, while the decoupled heads and recurrent update logic are more amenable to streaming execution. Third, the target XR deployment setting imposes tight latency and memory constraints, making it undesirable to duplicate full frame and event pipelines or to rely on repeated full-feature buffering.

These observations motivate the co-design strategy of HBTXR. The shared-backbone and decoupled-head model design allows a common compute substrate to be reused across modes, while the explicit Search/Track scheduler exposes operating states that can be mapped differently in terms of buffering, data reuse, and execution granularity.

B. CPU–FPGA Workload Partition

Figure 4 illustrates the overall deployment architecture. HBTXR adopts a heterogeneous CPU–FPGA organization in which the host processor handles irregular, control-centric tasks while the FPGA accelerates the latency-dominant neural inference path. Search/Track scheduling, state management, thresholding, and eye-state handling remain on the host because they are highly mode dependent. Frame/event tokenization, modality adaptation, shared-backbone inference, and Search/Event/Track/Mask heads are mapped to the FPGA, where they benefit from spatial parallelism and dedicated dataflow.

This partition is particularly important because the two modes interact differently with control overhead. Search benefits from explicit host-visible mode management and a refresh-oriented path, while Track should remain state persistent and lightweight to minimize end-to-end latency.

C. Mode-Aware Mixed-Granularity Execution Architecture

A key design choice of the HBTXR accelerator is to avoid a single execution granularity for the full pipeline. Instead, HBTXR adopts a mode-aware mixed-granularity architecture, shown in Fig. 4. Stages that require broader synchronization across tokens or branches are mapped to a *phase-level* execution path. This is appropriate for shared-transformer processing, branch coordination, and refresh-heavy Search execution. In contrast, stages that benefit from sequential locality and reuse are mapped to a *tile-stream* path. This path is more suitable for Track updates, lightweight head execution, and recurrent state propagation, where minimizing idle gaps and buffering overhead is more important than broad synchronization.

D. Shared Backbone Engine and Mode-Specific Head Mapping

The computational core of the accelerator is a shared transformer-oriented backbone engine. Rather than implementing separate hardware pipelines for frame Search and event Track, HBTXR maintains one shared execution substrate and attaches mode-specific heads on top of it. Let T_f and T_e denote the frame and event token sequences. The shared engine computes

$$H_f = \mathcal{B}(T_f), \qquad H_e = \mathcal{B}(T_e), \tag{20}$$

where $\mathcal{B}(\cdot)$ denotes the common transformer engine. In Search mode, the accelerator emphasizes frame-guided refresh and activates Search and Mask heads. In Track mode, it prioritizes event-conditioned update and the Track head, combining event features with the encoded previous state. This design avoids duplicating dominant compute blocks while preserving functional separation between modes.

E. Memory Organization and Reuse-Oriented Dataflow

Because HBTXR targets on-device XR operation, memory movement is often more critical than arithmetic count. The accelerator therefore distinguishes between *refresh-oriented buffering* and *stream-oriented reuse*. Search mode uses refresh buffers to support broader feature regeneration and relocalization. Track mode uses a streaming token queue and persistent state buffers so that recent context can be reused without rematerializing the whole inference path.

This organization is illustrated in Fig. 5. During Search, frame and event inputs are staged into a refresh path that regenerates shared features. During Track, the accelerator reuses persistent features and updates only the event-conditioned path together with the previous-state encoding. In this way, the dominant cost of Search is paid only when anchor refresh is needed.

F. Runtime Control and Scheduling Support

To support mode-aware execution, the accelerator is coupled with an FSM-based runtime controller that interprets host-side Search/Track decisions and dispatches the corresponding execution path. The controller manages three classes of signals: 1) mode-control signals from the host, 2) data-valid and buffer-status signals inside the accelerator, and 3) branch-confidence and output-ready signals for execution synchronization and state update. Unlike a conventional fixed-graph transformer accelerator, HBTXR must realize Search and Track as distinct operating states of the system.

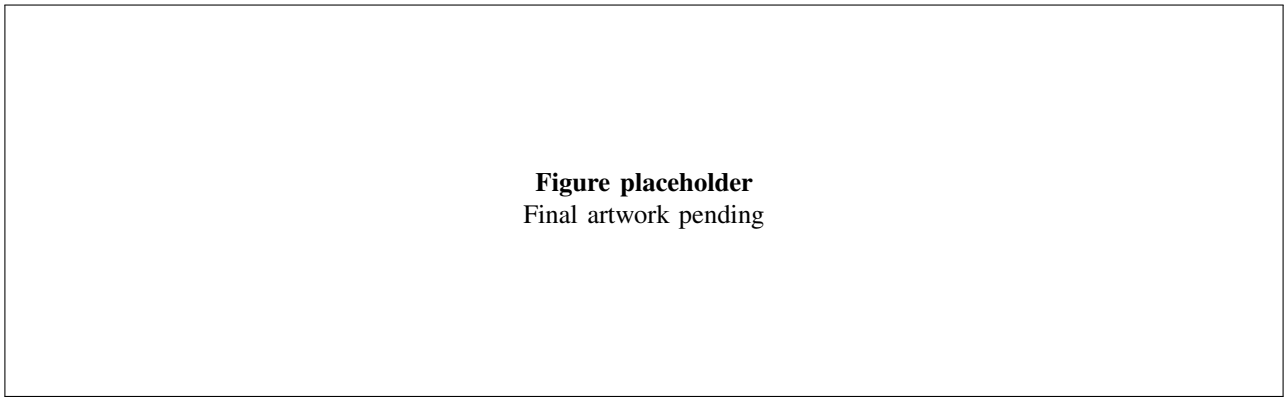

Fig. 4. Mode-aware mixed-granularity execution architecture of the proposed HBTXR accelerator. Phase-level execution is used for synchronization-heavy stages, whereas tile-stream execution is used for reuse-friendly update stages.

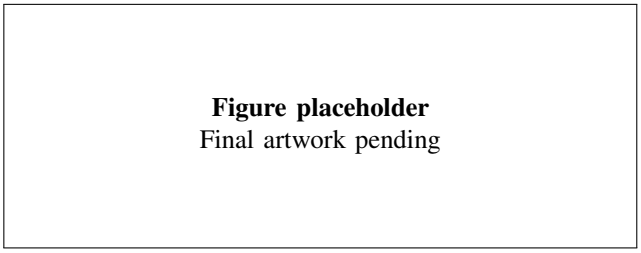

Fig. 5. Search-path and Track-path execution timelines. Search follows a refresh-heavy path with stronger synchronization, whereas Track follows a reuse-oriented path with persistent state and reduced idle gaps.

G. Latency Model and Deployment Scope

The end-to-end latency can be decomposed as

$$T_{e2e} = T_{host} + T_{transfer} + T_{exec} + T_{sync}, \tag{21}$$

where T_{host} denotes host-side control and scheduling, $T_{transfer}$ host-device communication, T_{exec} accelerator execution, and T_{sync} synchronization overhead. Because Search and Track have different execution structures, their latencies differ as well. Let T_{search} and T_{track} denote the corresponding mode-specific runtimes. The average runtime under scheduler-driven operation can be expressed as

$$T_{avg} = \alpha T_{search} + (1-\alpha) T_{track}, \tag{22}$$

where α is the fraction of Search invocations.

The purpose of the proposed accelerator is therefore not to maximize synthetic throughput in isolation, but to align execution behavior with the actual Search/Track operating pattern of HBTXR.

V. Experimental Results

A. Experimental Setup

Dataset and protocol: We evaluate HBTXR on a near-eye hybrid eye-tracking benchmark composed of synchronized frames, event streams, and dense pupil annotations. Each sample contains a canonicalized frame patch, a polarity-split event tensor, the previous tracking state, and geometry-aware targets including pupil mask, eye region, and current pupil state. A subject-disjoint split is used for training, validation, and final testing.

TABLE I
Experimental Setup Summary

Item	Setting
Training protocol	Two-stage Search pretrain + hybrid fine-tune
Input frame size	256×256
Input event size	$2 \times 256 \times 256$
Event policy	Fixed-count
Default event count	5000
Accumulation	Fast causal accumulation
Resize policy	Direct square canonical resize
Scheduler signals	Search conf., track conf., similarity, density, eye state
Optimizer	AdamW
Batch size	8
Learning rate	3×10^{-4}
Weight decay	1×10^{-4}
Runtime platform	Host-device heterogeneous execution

Training and runtime configuration: HBTXR is trained in two stages. The first stage optimizes Search-oriented relocalization, and the second stage jointly fine-tunes Search, Event, and Track branches with consistency and quality-aware gating. The default event representation uses a fixed-count window with 5000 events and fast causal accumulation. The runtime scheduler uses confidence, similarity, event density, and eye-state signals to choose between Search and Track.

Evaluation metrics: We report pupil-center error, gaze angular error, Search success rate, end-to-end latency, and effective update rate. Let $\hat{c}_t$ and c_t denote the predicted and ground-truth pupil centers. The average pupil-center error is

$$\mathrm{Err}_{pupil} = \frac{1}{N}\sum_{t=1}^{N} \|\hat{c}_t - c_t\|_2. \tag{23}$$

If $\hat{g}_t$ and g_t denote predicted and reference gaze vectors, the average gaze error is

$$\mathrm{Err}_{gaze} = \frac{1}{N}\sum_{t=1}^{N} \arccos\left(\frac{\hat{g}_t^\top g_t}{\|\hat{g}_t\|_2 \|g_t\|_2}\right). \tag{24}$$

B. Main Results

Table III summarizes the core performance of HBTXR. In Search mode, HBTXR achieves a pupil-center error of 0.58

TABLE II
BASELINE CATEGORIES USED IN COMPARISON

Category	Representative Methods	Primary Focus
Frame-based	Etracker, HE-Tracker	Semantic stability
Event-based	E-Track, FACET, Swift-Eye	Low-latency update
Hybrid / deploy.	EX-Gaze, SEE, EyeCoD, JaneEye	Fusion / deployment

Figure placeholder
Final artwork pending

Fig. 6. Qualitative tracking results of HBTXR under Search and Track modes. The examples show stable local updates in Track mode and robust recovery under degraded conditions through Search-mode re-entry.

TABLE III
MAIN RESULTS OF HBTXR

Mode	Pupil Err. (px)	Gaze Err. (deg)	Search SR (%)	Latency (ms)	Update Rate (Hz)
Search	0.58	0.91	97.8	0.93	1075
Track	**0.42**	**0.68**	–	**0.57**	**1754**
Scheduled Avg.	0.45	0.71	97.8	0.63	1587

Figure placeholder
Final artwork pending

Fig. 7. Latency breakdown of HBTXR. The total runtime is decomposed into host control, data transfer, accelerator execution, and synchronization overhead.

pixels, a gaze error of 0.91°, a Search success rate of 97.8%, and a latency of 0.93 ms. In Track mode, HBTXR reaches a pupil-center error of 0.42 pixels, a gaze error of 0.68°, and an end-to-end latency of 0.57 ms, corresponding to an effective update rate of 1754 Hz. Under scheduler-driven operation, the average runtime is reduced to 0.63 ms while preserving a pupil-center error of 0.45 pixels and a gaze error of 0.71°.

These results indicate that HBTXR preserves both geometric accuracy and sub-millisecond responsiveness during steady-state tracking. More importantly, the contrast between Search and Track validates the central design intuition of HBTXR: Search should serve as a recovery-oriented refresh path, whereas Track should dominate steady-state low-latency operation.

C. Comparison With Prior Eye-Tracking Methods

Table IV summarizes representative prior works using the metric form reported in their source papers. Because these methods differ in task formulation, datasets, runtime platforms, and evaluation protocol, the comparison is intentionally source-native rather than forcibly normalized. This avoids mixing pupil-localization error, gaze-estimation accuracy, and platform-specific timing definitions into unsupported percentage claims.

The comparison nevertheless highlights a clear trend. Frame-based systems such as Etracker and HE-Tracker report gaze quality but remain limited in operating rate. Event-only systems such as E-Track and FACET emphasize pupil localization and inference latency. EX-Gaze demonstrates the benefit of hybrid timing, but its reported pipeline still separates tracking and relocalization latency. In contrast, HBTXR combines explicit relocalization support, runtime adaptivity, and sub-millisecond Track latency within one unified Search/Track framework.

D. Deployment-Oriented Analysis

Table V reports deployment-oriented baselines in the form used by their original papers. We intentionally avoid converting energy-per-frame into power or frame rate into latency unless the source explicitly reports both. This prevents false equivalence across ASIC, FPGA SoC, and host-device systems.

The reported values indicate that EyeCoD targets 240 FPS and reports 154.32 mW silicon power at 370 MHz, SEE-D reports 0.70 ms latency and 2.29 mJ per inference, and JaneEye reports 0.5 ms latency, 2000 FPS, and 18.9 μJ/frame. These baselines confirm that low-latency eye tracking is feasible in specialized deployments, but they also show that deployment metrics remain strongly platform dependent. In this context, HBTXR is best positioned as a hybrid Search/Track system with runtime adaptivity rather than a normalized replacement for any single event-only accelerator baseline.

E. Ablation Studies

Table VI reports the ablation results. Removing geometry-stable supervision increases the pupil-center error from 0.42 to 0.49 pixels and the gaze error from 0.68° to 0.80°, confirming that the shared target geometry is essential for aligning Search and Track. Removing the decoupled head structure degrades both accuracy and latency, showing that functional specialization matters even when the backbone is shared.

The runtime-related ablations show a different pattern. Disabling the Search/Track scheduler increases latency from 0.57 ms to 0.71 ms while degrading gaze accuracy to 0.73°, indicating that runtime adaptivity improves both efficiency and stability. Disabling reuse-oriented execution leaves accuracy unchanged but increases latency to 0.69 ms, showing that feature reuse is primarily a deployment gain rather than an estimation gain.

F. Discussion

The qualitative results in Fig. 6 show that HBTXR maintains stable tracking during normal operation and recovers effectively when confidence degrades. The latency breakdown in Fig. 7 further indicates that the dominant practical gain comes from reducing repeated refresh cost during steady-state tracking.

TABLE IV
VERIFIED COMPARISON WITH PRIOR EYE-TRACKING METHODS USING SOURCE-REPORTED METRICS

Work	Mod.	Primary Task	Reported Accuracy	Reported Runtime	Reloc.	Note
Etracker	Frame	Gaze	0.53° gaze accuracy	30–60 Hz	Yes	Mobile near-eye gaze tracker
HE-Tracker	Frame	Gaze	3.655° gaze error (all subjects)	48 fps on AR HMD	Yes	Head-eye coordination model
E-Track	Event	Pupil	3.68 px median center-distance error	66.4 ms U-Net inference; 160 mW system	No	Event-only pupil tracking
FACET	Event	Pupil	0.2030 px PE	0.5302 ms inference	No	Event ellipse modeling
EX-Gaze	Hybrid	Pupil / tracking	1.33 px (saccade), 1.49 px (smooth pursuit) on EV-Eye	0.32 ms event tracking; 1.17 ms relocalization; $\sim$0.407 ms average op.	Yes	2 KHz hybrid system
HBTXR	Hybrid	Pupil + Gaze	0.42 px pupil err.; 0.68° gaze err.	0.57 ms Track mode; 1754 Hz	Yes	Unified Search/Track scheduler

TABLE V
VERIFIED DEPLOYMENT-ORIENTED COMPARISON USING SOURCE-REPORTED RUNTIME AND EFFICIENCY METRICS

System	Mod.	Runtime Adaptivity	Reported Runtime	Reported Efficiency	Deployment Scope
EyeCoD	Frame	No	240 FPS target throughput	154.32 mW silicon power at 370 MHz; 335 mW config table	FlatCam-based frame pipeline
SEE-D	Event	Partial	0.70 ms latency	2.29 mJ/inference	Event-based FPGA SoC pipeline
JaneEye	Event	No	0.5 ms; 2000 FPS	18.9 μJ/frame	Event-based 12-nm ASIC
HBTXR	Hybrid	Yes	0.57 ms Track mode; 1754 Hz	Not reported	Hybrid Search/Track host-device deployment

Figure placeholder
Final artwork pending

Fig. 8. Illustrative qualitative positioning of HBTXR and prior methods using source-reported metrics. Because the referenced works use heterogeneous tasks, datasets, and runtime definitions, the plot should be interpreted qualitatively rather than as a normalized ranking.

TABLE VI
ABLATION STUDY OF HBTXR

Variant	Pupil Err. (px)	Gaze Err. (deg)	Latency (ms)
Full HBTXR	0.42	0.68	0.57
w/o geom.-stable supervision	0.49	0.80	0.58
w/o decoupled heads	0.47	0.75	0.62
w/o Search/Track scheduler	0.46	0.73	0.71
w/o reuse-oriented execution	0.42	0.68	0.69

This observation is consistent with the central hypothesis of HBTXR: the system benefits not only from a stronger predictor, but also from executing Search and Track differently.

Overall, the experiments support three conclusions. First, hybrid sensing alone is insufficient unless Search and Track are explicitly separated at the algorithmic and runtime levels. Second, geometry-stable supervision is critical for making frame-guided relocalization and event-guided residual tracking compatible within one model. Third, deployment behavior is significantly improved by mode-aware mixed-granularity execution, which lowers the steady-state cost while preserving robust relocalization capability.

VI. CONCLUSION

This paper presented HBTXR, an algorithm–hardware co-designed hybrid eye-tracking system for on-device XR. The proposed framework is motivated by a simple but consequential observation: practical XR eye tracking cannot be solved adequately by frame-only or event-only pipelines in isolation. Frame-based methods provide semantically stable observations but incur high bandwidth and latency costs, whereas event-based methods provide fast and sparse sensing but remain weaker in relocalization and long-horizon robustness.

To bridge this gap, HBTXR formulates eye tracking as a hybrid Search-and-Track problem and jointly considers supervision, model design, runtime scheduling, and deployment mapping within one unified system framework. The resulting design combines geometry-stable hybrid supervision, a shared-backbone transformer with decoupled heads, a runtime-aware Search/Track scheduler, and a mode-aware deployment architecture with feature reuse and low-overhead control.

Experimental results demonstrate that HBTXR achieves a pupil-center error of 0.42 pixels, a gaze error of 0.68°, and an end-to-end latency of 0.57 ms in Track mode while maintaining a Search success rate of 97.8%. These results indicate that the main value of HBTXR lies in its integrated operating point: frame-guided Search improves recovery robustness, event-guided Track improves low-latency update, and deployment-aware execution makes the combined system practical on device.

Several directions remain open. The Search path can be optimized further for more aggressive refresh efficiency, the current deployment should be extended with deeper resource

and energy characterization, and the framework may be generalized to broader eye-related tasks such as blink understanding and user-state estimation. More broadly, HBTXR suggests that future XR eye tracking will benefit most from designs that co-optimize sensing, modeling, runtime control, and hardware execution together rather than treating them as isolated layers of the stack.

REFERENCES

[1] A. Plopski, T. Hirzle, N. Norouzi, L. Qian, G. Bruder, and T. Langlotz, "The eye in extended reality: A survey on gaze interaction and eye tracking in head-worn extended reality," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022.

[2] H. Kwon, K. Nair, J. Seo, J. Yik, D. Mohapatra, D. Zhan, J. Song, P. Capak, P. Zhang, and P. Vajda *et al.*, "Xrbench: An extended reality machine learning benchmark suite for the metaverse," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 1–20, 2023.

[3] B. Li, H. Fu, D. Wen, and W. Lo, "Etracker: A mobile gaze-tracking system with near-eye display based on a combined gaze-tracking algorithm," *Sensors*, vol. 18, no. 5, p. 1626, 2018.

[4] L. Chen, Y. Li, X. Bai, X. Wang, Y. Hu, M. Song, L. Xie, Y. Yan, and E. Yin, "Real-time gaze tracking with head-eye coordination for head-mounted displays," in *Proc. IEEE ISMAR*, 2022, pp. 82–91.

[5] N. Li, M. Chang, and A. Raychowdhury, "E-gaze: Gaze estimation with event camera," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 7, pp. 4796–4811, 2024.

[6] N. Li, A. Bhat, and A. Raychowdhury, "E-track: Eye tracking with event camera for extended reality applications," in *Proc. IEEE AICAS*, 2023, pp. 1–5.

[7] J. Ding, Z. Wang, C. Gao, M. Liu, and Q. Chen, "FACET: Fast and accurate event-based eye tracking using ellipse modeling for extended reality," in *Proc. IEEE ICRA*, 2025, pp. 10347–10354.

[8] P. Jain, A. M. Nazar, S. S. Khan, K. Mitra, and P. Chakravarthula, "Flat-track: Eye-tracking with ultra-thin lensless cameras," in *Proc. WACV*, 2025, pp. 433–441.

[9] Z. Wang, C. Gao, Z. Wu, M. V. Conde, R. Timofte, S.-C. Liu, Q. Chen, Z.-J. Zha, W. Zhai, H. Han *et al.*, "Event-based eye tracking: AIS 2024 challenge survey," in *Proc. IEEE/CVF CVPR*, 2024, pp. 5810–5825.

[10] Q. Chen, C. Gao, M. Liu, D. Perrone, Y. R. Pei, Z. Wang, Z. Zou, S. Tan, T. Han, G. Lu *et al.*, "Event-based eye tracking: Event-based vision workshop 2025," in *Proc. IEEE/CVF CVPR Workshops*, 2025, pp. 5164–5176.

[11] T. Zhang, Y. Shen, G. Zhao, L. Wang, X. Chen, L. Bai, and Y. Zhou, "Swift-eye: Towards anti-blink pupil tracking for precise and robust high-frequency near-eye movement analysis with event cameras," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 5, pp. 2077–2086, 2024.

[12] Q. Chen, Z. Wang, S.-C. Liu, and C. Gao, "3ET: Efficient event-based eye tracking using a change-based ConvLSTM network," in *Proc. IEEE BioCAS*, 2023, pp. 1–5.

[13] Y. Feng, N. Goulding-Hotta, A. Khan, H. Reyserhove, and Y. Zhu, "Real-time gaze tracking with event-driven eye segmentation," in *Proc. IEEE VR*, 2022, pp. 399–408.

[14] B. Lai, Z. Wang, X. Liu, Y. Wang, and X. Zhang, "An efficient eye tracking system based on event camera and SCNN-AKF fusion," *IEEE Photonics Technology Letters*, 2026.

[15] J. A. Leñero-Bardallo, T. Serrano-Gotarredona, and B. Linares-Barranco, "A 3.6μs latency asynchronous frame-free event-driven dynamic-vision sensor," *IEEE Journal of Solid-State Circuits*, vol. 46, no. 6, pp. 1443–1455, 2011.

[16] N. Chen, Y. Shen, T. Zhang, Y. Yang, and H. Wen, "EX-Gaze: High-frequency and low-latency gaze tracking with hybrid event-frame cameras for on-device extended reality," *IEEE Transactions on Visualization and Computer Graphics*, 2025.

[17] A. Sen, N. S. Bandara, I. Gokarn, T. Kandappu, and A. Misra, "EyeTraES: Fine-grained, low-latency eye tracking via adaptive event slicing," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 8, no. 4, pp. 1–32, 2024.

[18] Y. Zhao, X. Song, X. Huang, S. Tian, and Y. Zhou, "EV-GazeLock: A user authentication system based on micro eye movement with event cameras," *ACM Transactions on Sensor Networks*, 2025.

[19] Z. Sui, W. He, F. Liang, Y. Feng, X. Wei, Q. Lian, Z. Zhang, G. Li, and W. Wang, "Event-based low-power spiking gaze estimation," *Engineering Applications of Artificial Intelligence*, vol. 165, p. 113359, 2026.

[20] G. Zhao, Y. Yang, J. Liu, N. Chen, Y. Shen, H. Wen, and G. Lan, "EV-Eye: Rethinking high-frequency eye tracking through the lenses of event cameras," *Advances in Neural Information Processing Systems*, vol. 36, pp. 62169–62182, 2023.

[21] H. You, C. Wan, Y. Zhao, Z. Yu, Y. Fu, J. Yuan, S. Wu, S. Zhang, Y. Zhang, C. Li *et al.*, "EyeCoD: Eye tracking system acceleration via FlatCam-based algorithm and accelerator co-design," in *Proc. ISCA*, 2022, pp. 610–622.

[22] T. Han, A. Li, Q. Chen, and C. Gao, "JaneEye: A 12-nm 2K-FPS 18.9-μJ/frame event-based eye tracking accelerator," in *Proc. ASP-DAC*, 2026, pp. 170–176.

[23] M. Giordano, P. Bonazzi, L. Benini, and M. Magno, "Sub-millisecond event-based eye tracking on a resource-constrained microcontroller," in *Proc. IWASI*, 2025, pp. 1–6.

[24] P. Bonazzi, S. Bian, G. Lippolis, Y. Li, S. Sheik, and M. Magno, "Retina: Low-power eye tracking with event camera and spiking hardware," in *Proc. IEEE/CVF CVPR*, 2024, pp. 5684–5692.